



Linux Fundamentals

0%



Section 14 / 30

Permission Management

In Linux, permissions are like keys that control access to files and directories. These permissions are assigned to both users and groups, much like keys being distributed to specific individuals and teams within an organization. Each user can belong to multiple groups, and being part of a group grants additional access rights, allowing users to perform specific actions on files and directories.

Every file and directory has an owner (a user) and is associated with a group. The permissions for these files are defined for both the owner and the group, determining what actions—like reading, writing, or executing—are allowed. When you create a new file or directory, it automatically becomes "yours" and is associated with the group you belong to, similar to how a project within a company might default to your team's oversight.

In essence, Linux permissions act like a set of rules or keys that dictate who can access or modify certain resources, ensuring security and proper collaboration across the system.

When a user wants to access the contents of a Linux directory, it's similar to unlocking a door before stepping inside. To "traverse" or navigate into a directory, the user must first have the right

key—this key is the `execute` permission on the directory. Without it, even if the contents of the directory are visible to the user, they won't be able to enter or move through it.

In other words, having `execute` permissions on a directory is like having permission to walk through a hallway to access the rooms inside. It doesn't allow you to see or modify what's inside, but it does grant you the ability to step inside and explore the directory's structure. Without this permission, the user cannot access the directory's contents and will instead be presented with a "`Permission Denied`" error message.

shellsession

```
cry011t3@htb[/htb]$ ls -l
```

```
drw-rw-r-- 3 cry011t3 cry011t3 4096 Jan 12 12:30 scripts
```

```
cry011t3@htb[/htb]$ ls -al mydirectory/
```

```
ls: cannot access 'mydirectory/script.sh': Permission denied
```

```
ls: cannot access 'mydirectory/..': Permission denied
```

```
ls: cannot access 'mydirectory/subdirectory': Permission denied
```

```
ls: cannot access 'mydirectory/.': Permission denied
```

```
total 0
```

```
d????????? ? ? ? ? ? .
```

```
d????????? ? ? ? ? ? ..
```

```
-????????? ? ? ? ? ? script.sh
```

```
d????????? ? ? ? ? ? subdirectory
```

It is important to note that **execute** permissions are necessary to traverse a directory, no matter the user's level of access. Also, **execute** permissions on a directory do not allow a user to execute or modify any files or contents within the directory, only to traverse and access the content of the directory.

To execute files within the directory, a user needs **execute** permissions on the corresponding file. To modify the contents of a directory (create, delete, or rename files and subdirectories), the user needs **write** permissions on the directory.

The whole permission system on Linux systems is based on the octal number system, and basically, there are three different types of permissions a file or directory can be assigned:

- (**r**) - Read
- (**w**) - Write
- (**x**) - Execute

The permissions can be set for the **owner**, **group**, and **others** like presented in the next example with their corresponding permissions.

```
cry011t3@htb[/htb]$ ls -l /etc/passwd
```

```
- rwx rw- r--  1 root root 1641 May  4 23:42 /etc/passwd
- --- --- --- | | | | |
| | | | | | | | | _ Date
| | | | | | | | | File Size
| | | | | | | | | Group
| | | | | | | | | User
```

shellsession

```
| | | | _____ Number of hard links
| | | | _ Permission of others (read)
| | | _____ Permissions of the group (read, write)
| | _____ Permissions of the owner (read, write, execute)
| _____ File type (- = File, d = Directory, l = Link, ... )
```

Change Permissions

We can modify permissions using the `chmod` command, permission group references (`u` - owner, `g` - Group, `o` - others, `a` - All users), and either a `+` or a `-` to add or remove the designated permissions. In the following example, let us assume we have a file called `shell` and we want to change permissions for it so this script is owned by that user, becomes not executable, and set with read/write permissions for all users.

```
cry011t3@htb[/htb]$ ls -l shell
```

```
-rwxr-x--x  1 cry011t3 htbteam 0 May  4 22:12 shell
```

shellsession

We can then apply `read` permissions for all users and see the result.

```
cry011t3@htb[/htb]$ chmod a+r shell && ls -l shell
```

```
-rwxr-xr-x  1 cry011t3 htbteam 0 May  4 22:12 shell
```

shellsession

We can also set the permissions for all other users to **read** only using the octal value assignment.

```
shellsession
cry011t3@htb[/htb]$ chmod 754 shell && ls -l shell
-rwxr-xr--  1 cry011t3 htbteam 0 May  4 22:12 shell
```

Let us look at all the representations associated with it to understand better how the permission assignment is calculated.

```
shellsession
Binary Notation:           4 2 1 | 4 2 1 | 4 2 1
-----
Binary Representation:     1 1 1 | 1 0 1 | 1 0 0
-----
Octal Value:               7   | 5   | 4
-----
Permission Representation:  r w x | r - x | r - -
```

If we sum the set bits from the **Binary Representation** assigned to the values from **Binary Notation** together, we get the **Octal Value**. The **Permission Representation** represents the bits set in the **Binary Representation** by using the three characters, which only recognizes the set permissions easier.

Change Owner

To change the owner and/or the group assignments of a file or directory, we can use the `chown` command. The syntax is like following:

Syntax - chown

```
cry011t3@htb[/htb]$ chown <user>:<group> <file/directory>
```

shellsession

In this example, "shell" can be replaced with any arbitrary file or folder.

```
cry011t3@htb[/htb]$ chown root:root shell && ls -l shell
```

```
-rwxr-xr--  1 root root 0 May  4 22:12 shell
```

shellsession

SUID & SGID

In addition to standard user and group permissions, Linux allows us to configure special permissions on files through the Set User ID (`SUID`) and Set Group ID (`SGID`) bits. These bits function like temporary access passes, enabling users to run certain programs with the privileges of another user or group. For example, administrators can use `SUID` or `SGID` to grant users elevated rights for specific applications, allowing tasks to be performed with the necessary permissions, even if the user themselves doesn't normally have them.

The presence of these permissions is indicated by an **s** in place of the usual **x** in the file's permission set. When a program with the SUID or SGID bit set is executed, it runs with the permissions of the file's owner or group, rather than the user who launched it. This can be useful for certain system tasks but also introduces potential security risks if not used carefully.

One common risk is when administrators, unfamiliar with an application's full functionality, assign **SUID** or **SGID** bits indiscriminately. For example, if the **SUID** bit is applied to a program like **journalctl**, which includes a function to launch a shell from within its interface, any user running this program could execute a shell as root. This grants them complete control over the system, presenting a significant security vulnerability. More information about this and other such applications can be found at [GTFObins](#).

Sticky Bit

Sticky bits in Linux are like locks on files within shared spaces. When set on a directory, the sticky bit adds an extra layer of security, ensuring that only certain individuals can modify or delete files, even if others have access to the directory.

Imagine a communal workspace where many people can enter and use the same tools, but each person has their own drawer that only they (or the manager) can open. The sticky bit acts like a lock on these drawers, preventing anyone else from tampering with the contents. In a shared directory, this means only the file's owner, the directory's owner, or the root user (the system administrator) can delete or rename files. Other users can still access the directory but can't modify files they don't own.

This feature is especially useful in shared environments, like public directories, where multiple users are working together. By setting the sticky bit, you ensure that important files aren't accidentally or maliciously altered by someone who shouldn't have the authority to do so, adding an important safeguard to collaborative workspaces.

```
cry011t3@htb[/htb]$ ls -l
```

shellsession

```
drw-rw-r-t 3 cry011t3 cry011t3 4096 Jan 12 12:30 scripts
drw-rw-r-T 3 cry011t3 cry011t3 4096 Jan 12 12:32 reports
```

In this example, we see that both directories have the sticky bit set. However, the **reports** folder has an uppercase **T**, and the **scripts** folder has a lowercase **t**.

If the sticky bit is capitalized (**T**), then this means that all other users do not have **execute** (**x**) permissions and, therefore, cannot see the contents of the folder nor run any programs from it. The lowercase sticky bit (**t**) is the sticky bit where the **execute** (**x**) permissions have been set.

Connect to HTB



14 / 30 Sections



Mark Complete & Next